

Numeric Data Types

Section 1

Chapter 2

An Introduction to Programming Using Python
David I. Schneider

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Numbers

- Numbers are referred to as *numeric literals*
- Two Types of numbers: *ints* and *floats*
 - Whole number written without a decimal point called an **int** e.g. 3, 0, -17, 549 etc.
 - Number written with a decimal point called a **float** e.g. 3.0, 5.329, 0.0, -23.45 etc.

Numbers

- Arithmetic Operators
 - Addition, subtraction, multiplication, division, and exponentiation.
- Result of a division is always a float
- Result of the other operations is a float if ...
 - Either of the numbers is a float
 - Otherwise is an int.

Operators and Operands

- Operators are special **tokens** that represent computations.
 - Arithmetic Computations:
 - Addition, subtraction, multiplication, division, and exponentiation.
- The values the operator uses are called **operands**.
- Examples: $2+2$, $5/3$, $5-1$, $7*7$

Basic Arithmetic Operators

Common operators in Python are as follows:

- `+` : addition
- `-` : subtraction
- `*` : multiplication
- `/` : division (Result of a division is always a float)
- `**` : exponentiation
- Result of the other operations is a float if ...
 - Either of the numbers is a float
 - Otherwise is an int.

Additional Operators

- `//` : integer division (divides and returns the quotient). Also called floor division.
- `%` : modulus (remainder operator)
- `=` : assignment operator (i.e. `a = 4`)
- `==` : equality operator (i.e. if (`a == 4`) then do something)

$$\begin{array}{r} 4 \\ 3 \overline{)14} \\ \underline{12} \\ 2 \end{array} \quad \begin{array}{l} \leftarrow 14 // 3 \\ \\ \leftarrow 14 \% 3 \end{array}$$

Integer Division Example

- Example 5: converts 41 inches to 3 feet, 5 inches:

```
totalInches = 41
feet = totalInches // 12
inches = totalInches % 12
print(feet, inches)
```

[Run]

3 5

Parentheses, Order of Precedence

- Order of precedence
 1. Terms inside parentheses (inner to outer)
 2. Exponentiation
 3. Multiplication, division (ordinary and integer), modulus
 4. Addition and subtraction
 5. Operators with *same* precedence are evaluated left to right.
 1. An exception to this rule is exponentiation operator (**)
- Use parentheses to clarify meaning of an expression

The `abs` Function

Expression	Value
<code>abs(3)</code>	3
<code>abs(0)</code>	0
<code>abs(-3)</code>	3

The `int` Function

Expression	Value
<code>int(2.7)</code>	2
<code>int(3)</code>	3
<code>int(-2.7)</code>	-2

The `round` Function

Expression	Value
<code>round(2.7)</code>	3
<code>round(2.317, 2)</code>	2.32
<code>round(2.317, 1)</code>	2.3

abs, int, and round Example

```
a = 2
b = 3
print(abs(1 - (4 * b)))
print(int((a ** b) + .8))
print(round(a / b, 3))
```

[Run]

11

8

0.667

Variables and Expressions

- Names given to the values are called **variables**
- **Expression:** A combination of values, variables, operators and function calls that can be evaluated.
- The evaluation of an **expression** produces a **value**.
 - A **value** by itself is a simple expression.
 - A **variable** by itself is a simple expression.

Variables and Expressions

```
speed = 50
timeElapsed = 14
distance = speed * timeElapsed
print(distance)
```

[Run]

700

```
a = 2
b = 3
print(abs(1 - (4 * b)))
print(int((a ** b) + .8))
print(round(a / b, 3))
```

[Run]

11

8

0.667

Syntax Errors

- Grammatical and punctuation errors are called syntax errors.
 - Typically found **before** a program is run.

<i>Statement</i>	<i>Reason for Error</i>
<code>print(3))</code>	The statement contains an extraneous right parenthesis.
<code>for = 5</code>	A reserved word was used as a variable name.
<code>print(2; 3)</code>	The semicolon should be a comma.

Table 2.1 Three Syntax Errors

Syntax Errors

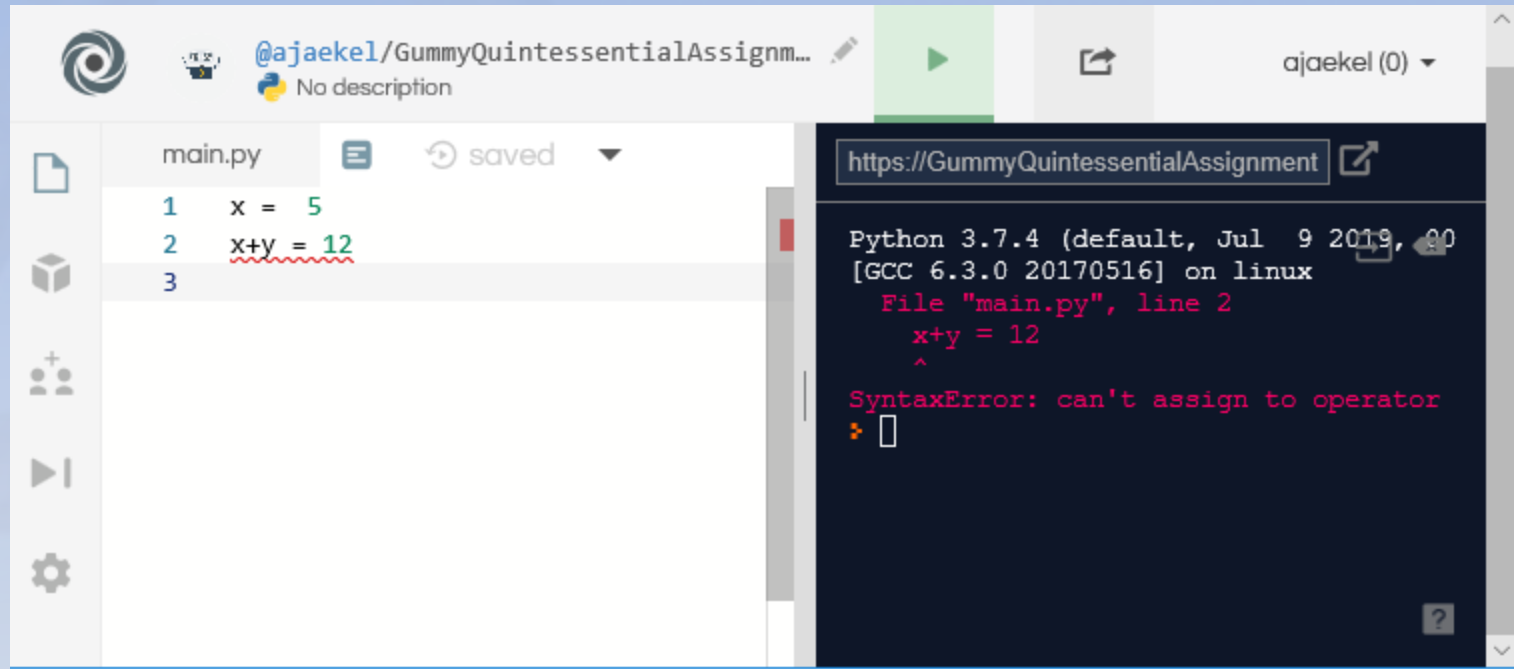


FIGURE 2.1 Syntax error message boxes.

Runtime Errors

- Errors discovered while program is running called runtime errors or exceptions

<i>Statement</i>	<i>Reason for Error</i>
<code>printt (5)</code>	The function <code>print</code> is misspelled.
<code>x += 1</code> , when <code>x</code> has not been created	Python is not aware of the variable <code>x</code> .
<code>print(5 / 0)</code>	A number cannot be divided by zero.

Table 2.2 Three Runtime Errors

Runtime Errors

- When Python encounters exception
 - Terminates execution of program, displays message

```
Traceback (most recent call last):  
  File "C:\test1.py", line 2, in <module>  
    print(5 / 0) #ZeroDivisionError  
ZeroDivisionError: division by zero
```

FIGURE 2.2 An Error Message for an Exception Error

Logic Error

- Occurs when a program does not perform the way it was intended
- Example
$$\text{average} = \text{firstNum} + \text{secondNum} / 2$$
- Syntax correct, logic wrong: should be
$$\text{average} = (\text{firstNum} + \text{secondNum}) / 2$$
- Logic errors are most difficult type of error to locate

End

Section 1

Chapter 2

An Introduction to Programming Using Python
David I. Schneider

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.